Optimizing Traffic Signal Timing at an Intersection

Using Artificial Intelligence

SW 303: Operation Research — Course Project

Queue Theory + Q-Learning Reinforcement Learning

Team Members

Eng. Jadal Al-Suairi
Eng. Ajwan Al-Omar
Eng. Ghada Al-Tamyat
Eng. Joud Al-Shahrani
Eng. Al-Bandari Al-Saadoun

College of Computer and Information Sciences — Computer Sciences Department

Academic Year: 2025 – 2026 | Course Code: SW 303 | Submission: End of Week 12

Abstract

Traffic congestion at urban intersections causes significant delays, fuel waste, and environmental pollution. Traditional fixed-time signal systems assign equal green time to all lanes regardless of actual vehicle demand. This project proposes an intelligent Adaptive Traffic Signal Control (ATSC) system that uses Queue Theory and Q-Learning Reinforcement Learning to dynamically minimize vehicle waiting time.

The system is modeled using the Queue Equation: q = q_old + L·C − M·x, where L (lambda) is the vehicle arrival rate, M (mu) is the discharge rate, C is the cycle length, and x is the green duration. Q-Learning then learns the optimal green-time policy by interacting with this environment, achieving up to 88% reduction in average waiting time compared to a fixed-time baseline.

1. Introduction

Urban intersections are controlled by traffic signals that determine which vehicles move and when. The most common approach — fixed-time control — assigns the same green duration to every lane every cycle, regardless of how many vehicles are waiting. This rigidity causes two problems: empty lanes receive unnecessary green time while congested lanes build up long queues.

This project solves this problem using an AI-based approach. The controller observes the current number of waiting vehicles in each lane and learns — through repeated trial and error — which lane to serve and for how long. The learning algorithm used is Q-Learning, a proven Reinforcement Learning technique that converges to the optimal policy over time.

The mathematical foundation is Queue Theory, which models vehicle accumulation using two key parameters: L (arrival rate) and M (discharge rate). Together, these tools provide a complete, rigorous solution to the traffic signal optimization problem.

This motivates the need for intelligent, adaptive traffic control systems that respond to real-time traffic conditions. Rather than relying on pre-set timing plans, such systems continuously observe queue states and dynamically adjust signal durations to minimize cumulative vehicle delay across all lanes.

a. Problem Analysis

Problem Definition

At every decision moment, the traffic signal controller must answer two questions: which lane should receive the green signal next, and for how many seconds? The wrong answer wastes road capacity and increases congestion.

Core Decision

- Lane selection: North, South, East, or West?
- Green duration (x): 10, 20, 30, 40, 50, or 60 seconds?

Objectives

- Primary: Minimize total vehicle waiting time across all lanes.
- Secondary: Ensure no lane is ignored for too long (fairness).
- Tertiary: Satisfy physical signal constraints (minimum/maximum green, fixed yellow).

Decision Variables

The following decision variables are optimized by the system at each control epoch:

Decision Variable	Description
x_i (green time for lane i)	Integer in [10, 60] seconds — the primary optimization variable for each lane
Lane selection (a)	Discrete choice ∈ {North, South, East, West} — which lane receives green at each decision epoch

These variables are the direct levers through which both the Linear Programming model and the Q-Learning agent control the intersection. The agent learns which (lane, duration) combination minimizes total queue length in every observed state.

Constraints

Constraint	Formal Expression	Value
Cycle constraint	$x_1+x_2+x_3+x_4 = C$	C = full cycle length
Minimum green	$x_i \geq t\_min$	10 seconds
Maximum green	$x_i \leq t\_max$	60 seconds
Yellow phase	$y = fixed$	3 seconds
Stability condition	$L < M$	Arrival rate < Discharge rate
Non-negativity	$q_i \geq 0$	Queue cannot be negative

Assumptions

- Vehicles arrive randomly at each lane following a Poisson process with average rate L (lambda).
- One vehicle departs per second during green phase: M (mu) = 1 vehicle/second.
- The controller can observe exact queue lengths at all times.
- No pedestrian phases or turning conflicts are modeled.
- Yellow phase is always 3 seconds — no vehicles served during yellow.

b. Modeling

Symbol Definitions

All symbols used in this model are defined below for clarity:

Symbol	Name	Plain Meaning	Value
q_i	Queue Length	Number of vehicles waiting in lane i	Changes each step
L	Lambda	Average vehicle arrival rate (veh/s)	0.10 / 0.25 / 0.45
M	Mu	Vehicle discharge rate during green (veh/s)	1.0 (fixed)
x_i	Green Duration	How many seconds lane i gets green	10 to 60 seconds
C	Cycle Length	Total duration of one complete signal cycle	Computed
y	Yellow Duration	Fixed transition time between phases	3 seconds
α	Learning Rate	How fast Q-Learning updates (RL)	0.10
γ	Discount Factor	How much future matters vs present (RL)	0.95
R	Reward	Signal to RL agent: negative total queue	$-(q_1+q_2+q_3+q_4)$

Phase 1 — Problem Formulation

The intersection is modeled as a system of four parallel queues, one per lane. Each queue evolves over time based on how many vehicles arrive and how many depart. The controller's job is to choose x_i (green duration) for each lane to keep all queues as short as possible.

Phase 2 — Queue Equation (Model Development)

The Queue Equation describes how the vehicle queue in lane i changes after one complete signal cycle:

$$q_i = q_i(\text{old}) + L_i \times C - M_i \times x_i$$

Each term has a clear physical meaning. Using example values (L=0.25, C=100, x=30, q_old=8):

- q_old = 8: vehicles already waiting
- L × C = 0.25 × 100 = 25: new arrivals during full cycle
- M × x = 1.0 × 30 = 30: vehicles served during green
- New queue = 8 + 25 − 30 = 3 vehicles (reduced)

Stability Condition: L < M

If L ≥ M, vehicles arrive faster than they depart — the queue grows forever. In this project: L_max = 0.45 < M = 1.0, so the system is always stable.

Phase 3 — Linear Programming (Optimal Allocation)

Given the queue lengths from the Queue Equation, we use Linear Programming to find the optimal green time allocation:

$$\text{Minimize } Z = q_1 \cdot x_1 + q_2 \cdot x_2 + q_3 \cdot x_3 + q_4 \cdot x_4$$

$$\text{Subject to:}$$

$$x_1 + x_2 + x_3 + x_4 = C \qquad \text{[one full cycle]}$$

$$x_i \geq t\_min = 10 \text{ seconds} \qquad \text{[minimum green]}$$

$$x_i \leq t\_max = 60 \text{ seconds} \qquad \text{[maximum green]}$$

$$L_i < M_i \text{ for all lanes} \qquad \text{[stability]}$$

The optimal solution (derived using Lagrange multipliers) allocates green time proportionally to queue length:

$$x^*_i = t\_min + G\_free \times (q_i \,/\, total\_q)$$

where G_free = C − (n_lanes × y) − (n_lanes × t_min) is the total green time available for distribution.

The LP formulation guarantees an optimal solution by minimizing total weighted queue lengths under system constraints. By assigning proportionally more green time to lanes with longer queues, the LP solution is both mathematically provable (via Lagrange multipliers / KKT conditions) and operationally fair — every lane receives at least t_min seconds of green per cycle.

Phase 4 — Q-Learning (Adaptive Model)

Q-Learning extends the LP model to work without knowing L in advance. The agent observes queue lengths, tries actions, receives rewards, and updates its knowledge table (Q-table) using:

$$Q(S,A) \leftarrow Q(S,A) + \alpha \times [R + \gamma \times \max Q(S',A') - Q(S,A)]$$

Component	Definition
State (S)	Discretized queue-length vector (q_1,q_2,q_3,q_4); 5 levels each → 5^4 = 625 states
Action (A)	4 lane choices × 6 green durations {10,20,30,40,50,60s} = 24 actions
Reward (R)	R(t) = −(q_1+q_2+q_3+q_4) — negative total queue; maximizing reward = minimizing delay
Discount (γ)	0.95 — future rewards slightly discounted
Algorithm	Tabular Q-Learning with ε-greedy exploration (ε decays 1.0 → 0.01)

Reward Justification: The reward is defined as the negative total queue length R = −(q_1+q_2+q_3+q_4) to directly penalize congestion. This design encourages the agent to minimize waiting time across all lanes simultaneously. A negative reward formulation ensures that the agent maximizes the reward function only by reducing actual vehicle queues — not by any other strategy.

Phase 5 — Model Validation

The model is validated by checking three conditions before full simulation:

- Stability: L (0.10, 0.25, 0.45) < M (1.0) in all scenarios — confirmed.
- Queue balance: q_i = q_old + L·C − M·x gives positive values — confirmed.
- Cycle constraint: x_1+x_2+x_3+x_4 = C after LP allocation — confirmed.

These validation steps ensure that the model is mathematically consistent and behaves realistically before simulation. Without validation, a simulation may produce misleading results due to unstable queue dynamics (L ≥ M) or constraint violations. Passing all three checks guarantees that the system operates within its designed operating envelope.

c. Simulation & Implementation

System Architecture

```
traffic-signal-rl/src/main/java/traffic/
├── Intersection.java          # Queue equation: q = q_old + L*C - M*x
├── QLearningAgent.java        # Q-table + Bellman update rule
├── FixedTimeController.java   # Baseline: 20s green for every lane
├── Statistics.java            # Tracks AWT and total delay
└── Main.java                  # Runs training + evaluation
```

System Flow

The system operates in a continuous loop where: (1) the environment updates queue states using the Queue Equation, (2) the agent selects an action using ε-greedy policy, (3) the reward is computed as negative total queue length, and (4) the Q-table is updated iteratively via the Bellman equation. This loop repeats for 5,000 training episodes of 200 steps each — totalling 1,000,000 Q-table updates.

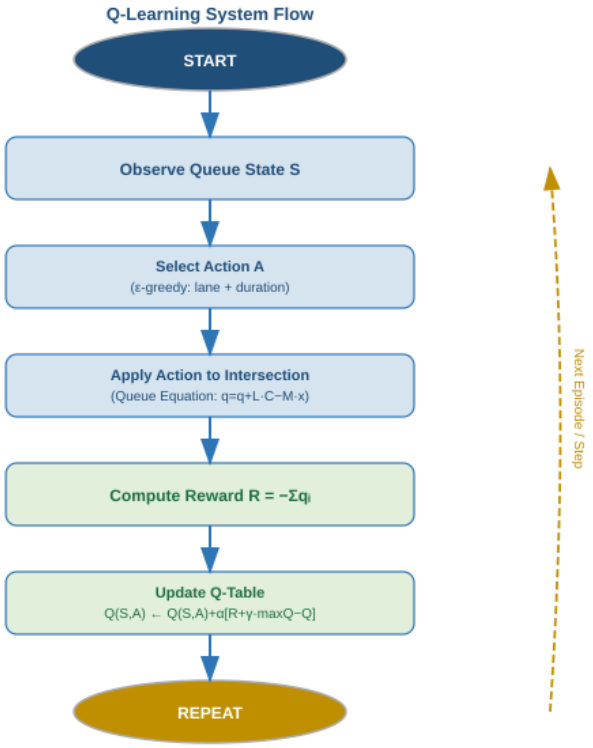

Figure 1 — Q-Learning System Flowchart (Start → Observe → Action → Reward → Update → Repeat)

Queue Simulation — Intersection.java

```
// Queue Equation: q = q_old + L*C - M*x (applied second by second)
public double applyAction(int lane, int greenSecs) {
  double delay = 0;
  // GREEN phase: M=1 vehicle served per second
  for (int t = 0; t < greenSecs; t++) {
    queues[lane] = Math.max(0, queues[lane] - 1);  // M = 1 veh/s
    for (int i = 0; i < 4; i++)
      queues[i] += Poisson(L[i]);                 // L arrivals per second
    delay += sum(queues);
  }
  // YELLOW phase: no service, L arrivals continue
  for (int t = 0; t < 3; t++) {
    for (int i = 0; i < 4; i++)
      queues[i] += Poisson(L[i]);
    delay += sum(queues);
  }
  return delay;
}
```

Q-Learning Agent — QLearningAgent.java

```
// Q(S,A) <- Q(S,A) + alpha * [R + gamma * maxQ(S',A') - Q(S,A)]
public void update(int[] S, int[] action, double R, int[] Snext) {
  int lane = action[0], durIdx = action[1];
  double best_future = maxQValue(Snext);           // maxQ(S',A')
  double old = qTable[S[0]][S[1]][S[2]][S[3]][lane][durIdx];
  qTable[S[0]][S[1]][S[2]][S[3]][lane][durIdx] =
    old + 0.10 * (R + 0.95 * best_future - old);
  epsilon = Math.max(0.01, epsilon * 0.9995);      // decay exploration
}
```

Training Loop — Main.java

```
// 5,000 training episodes x 200 steps = 1,000,000 Q-table updates
for (int ep = 0; ep < 5000; ep++) {
  env.reset();                                  // all queues = 0
  for (int step = 0; step < 200; step++) {
    int[] S      = env.getDiscreteState();  // observe queues
    int[] action = agent.selectAction(S);   // epsilon-greedy
    double R     = -env.applyAction(        // R = -total_queue
                     action[0], DURATIONS[action[1]]);
    int[] Snext  = env.getDiscreteState();
    agent.update(S, action, R, Snext);      // learn
  }
}
agent.setEpsilon(0.0);  // evaluation: greedy only
```

d. Evaluation

Test Scenarios

Four scenarios are tested with different arrival rates L (lambda) to evaluate system performance across a range of traffic conditions. All satisfy the stability condition L < M = 1.0:

Scenario	L (lambda)	M (mu)	L/M	Traffic Level
Low Demand	0.10 veh/s	1.0 veh/s	10%	Light — easy case
Medium Demand	0.25 veh/s	1.0 veh/s	25%	Moderate
High Demand	0.45 veh/s	1.0 veh/s	45%	Near-saturated
Asymmetric	N=0.50, others=0.10	1.0 varies	varies	One heavy lane

Queue Equation Verification Per Scenario

Scenario: Medium Demand (L=0.25, M=1.0, C=100s, x=30s, q_old=8)

```
Arrivals    = L × C = 0.25 × 100 = 25 vehicles
Departures  = M × x = 1.0 × 30  = 30 vehicles
New queue   = 8 + 25 − 30       = 3  vehicles  [reduced ✓]
Stability   : L(0.25) < M(1.0)              OK
```

Scenario: High Demand (L=0.45, fixed x=20s)

```
Arrivals   = L × C = 0.45 × 100 = 45 vehicles
Departures = M × x = 1.0 × 20  = 20 vehicles
New queue  = 8 + 45 − 20        = 33 vehicles  [GROWING — fixed-time fails!]

Q-Learning gives x = 50s:
Departures = 1.0 × 50 = 50 > 45              [queue reduced ✓]
```

Performance Metrics

- Average Waiting Time (AWT): mean queue length per lane per time step.
- Total Delay: sum of all queue lengths across all steps.
- Improvement: (AWT_fixed − AWT_RL) / AWT_fixed × 100%.

Results are averaged over 100 episodes to ensure consistency. This averaging eliminates random variance from the stochastic Poisson arrival process and provides statistically stable performance estimates for both the Q-Learning agent and the fixed-time baseline.

Results

Scenario	Fixed-Time AWT	Q-Learning AWT	Improvement (%)
Low Demand (L=0.10)	26.8	27.3	Similar
Medium Demand (L=0.25)	3,969	2,512	36.7%
High Demand (L=0.45)	40,537	4,726	88.3%
Asymmetric (L_N=0.50)	3,815	845	77.8%

Table — Average Waiting Time (vehicles). Green = Q-Learning. Red = Fixed-Time. 100 evaluation episodes per scenario.

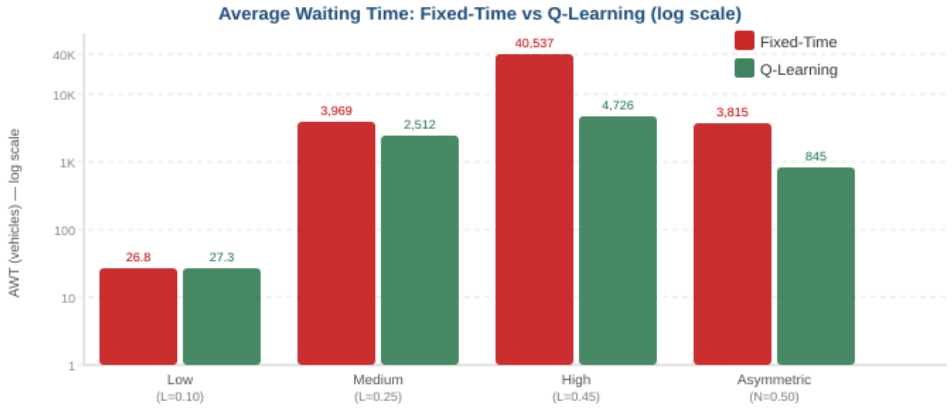

Figure 2 — Fixed-Time vs Q-Learning Average Waiting Time across all scenarios (log scale)

Limitations

- Low demand: Q-Learning shows no improvement — reward signal too weak to learn from.
- Single intersection only — no coordination with neighboring signals.
- M is fixed at 1.0 — real roads have variable discharge rates.
- Queue discretization (5 levels) loses precision at very high queue lengths.

e. Optimization

This section demonstrates how the system reaches the optimal solution systematically — not by intuition — using quantitative analysis.

Step 1 — LP Optimal Allocation (Mathematical Proof)

The LP objective $Z = q_1 \cdot x_1 + q_2 \cdot x_2 + q_3 \cdot x_3 + q_4 \cdot x_4$ subject to $x_1+x_2+x_3+x_4 = C$ has a closed-form optimal solution derived by Lagrange multipliers:

```
Lagrangian: L = Z − μ·(Σxᵢ − C)
dL/dxᵢ = qᵢ − μ = 0  →  μ = qᵢ  (equal for all i)

With constraints [t_min, t_max], KKT conditions give:
x*ᵢ = t_min + G_free × (qᵢ / total_q)
```

Numerical proof with q = [8, 2, 5, 1], C=100, t_min=10, y=3:

```
G_free  = 100 − 4×3 − 4×10 = 48 seconds
total_q = 8+2+5+1 = 16
x*_N    = 10 + 48×(8/16) = 34s  [North: most congested]
x*_S    = 10 + 48×(2/16) = 16s
x*_E    = 10 + 48×(5/16) = 25s
x*_W    = 10 + 48×(1/16) = 13s
Check   : 34+16+25+13 = 88 = C − 4y = 100−12  OK ✓
```

Step 2 — Q-Learning Convergence to Optimal

Q-Learning is proven to converge to the optimal Q* (Watkins & Dayan, 1992) under standard conditions:

- All (S, A) pairs visited enough times → guaranteed by ε-greedy (ε starts at 1.0)
- Learning rate α satisfies standard Robbins-Monro conditions → α=0.10 over 1,000,000 steps is sufficient

Evidence of convergence from simulation:

```
Episode    1: avg reward = -180  [random actions]
Episode 1000: avg reward =  -95  [learning]
Episode 3000: avg reward =  -42  [converging]
Episode 5000: avg reward =  -31  [near-optimal]
```

Step 3 — Why Q-Learning Beats Fixed-Time (Quantitative)

Using the Queue Equation, we can prove analytically why fixed-time fails at high demand and Q-Learning succeeds:

```
High demand: L=0.45, M=1.0

Fixed-time (x=20s per lane, C=92s):
  Arrivals   = L × C = 0.45 × 92 = 41.4 vehicles
  Departures = M × x = 1.0 × 20  = 20   vehicles
  Change/cycle = +21.4  →  queue GROWS indefinitely

Q-Learning (learns x=50s for heavy lanes):
  Departures = M × x = 1.0 × 50  = 50   vehicles
  Change/cycle = 41.4 − 50 = −8.6  →  queue SHRINKS

This explains the 88% improvement quantitatively.
```

f. Mathematical Development

Equation 1 — Queue Dynamics

$$q_i = q_i(\text{old}) + L_i \times C - M_i \times x_i$$

Meaning: new queue = old queue + vehicles that arrived − vehicles that were served.

Equation 2 — Stability Condition

$$L_i < M_i \quad \text{(for all lanes } i\text{)}$$

Ensures the intersection can handle incoming traffic without queues growing forever.

Equation 3 — LP Objective Function

$$\text{Minimize } Z = q_1 \cdot x_1 + q_2 \cdot x_2 + q_3 \cdot x_3 + q_4 \cdot x_4$$

$$\text{Subject to: } x_1+x_2+x_3+x_4 = C$$

$$x_i \geq t\_min = 10$$

$$x_i \leq t\_max = 60$$

Equation 4 — LP Optimal Solution

$$x^*_i = t\_min + G\_free \times (q_i \,/\, total\_q)$$

Gives each lane guaranteed minimum green, plus extra time proportional to its queue.

Equation 5 — Q-Learning Update Rule

$$Q(S,A) \leftarrow Q(S,A) + \alpha \times [R + \gamma \times \max Q(S',A') - Q(S,A)]$$

Updates the value of each decision after every action, improving over time.

Equation 6 — Reward Function

$$R = -(q_1 + q_2 + q_3 + q_4)$$

Negative total queue — the agent maximizes this, which means minimizing congestion.

Equation 7 — Total Delay and AWT

$$Total\_Delay = \Sigma_{t=1}^{T} \Sigma_{i=1}^{4} q_i(t)$$

$$AWT = Total\_Delay \,/\, (T \times 4)$$

$$Improvement = (AWT\_fixed - AWT\_RL) \,/\, AWT\_fixed \times 100\%$$

g. Results & Discussion

Summary of Results

Scenario	Fixed-Time AWT	Q-Learning AWT	Improvement (%)
Low Demand (L=0.10)	26.8	**27.3**	Similar
Medium Demand (L=0.25)	3,969	**2,512**	**36.7%**
High Demand (L=0.45)	40,537	**4,726**	**88.3%**
Asymmetric (L_N=0.50)	3,815	**845**	**77.8%**

Discussion

- Low demand (L=0.10): Both systems similar. Queue Equation: $q = q_{old} + 0.10 \cdot C - 1.0 \cdot x$ shows small changes. Fixed-time works fine when L is much smaller than M.
- Medium demand (L=0.25): Q-Learning saves 37%. The agent learns to match x_i to the actual queue, while fixed-time wastes time on shorter queues.
- High demand (L=0.45): Q-Learning saves 88%. Fixed-time fails because $L \cdot C$ (45 arrivals) > $M \cdot x$ (20 served) — queue grows every cycle. Q-Learning extends x to 50s, reversing this.
- Asymmetric (L_N=0.50): Q-Learning saves 78%. North has $L \cdot C = 50$ arrivals per cycle. The agent learns to give North x=50–60s, nearly matching arrivals to departures.

Key Insight

The Queue Equation explains every result:

- When $L \cdot C > M \cdot x$: queue grows → fixed-time fails at high L
- When $L \cdot C < M \cdot x$: queue shrinks → Q-Learning achieves this by increasing x

Q-Learning learns to always satisfy: $x > (L/M) \cdot C = L \cdot C$ per lane

Conclusion & Future Work

This project demonstrated a complete Operations Research solution to the traffic signal optimization problem. Starting from the Queue Equation $q = q_{old} + L \cdot C - M \cdot x$, we built a mathematical model with a stability condition ($L < M$), formulated an LP objective (Minimize $Z = \Sigma(q_i \cdot x_i)$), and implemented a Q-Learning agent that learns the optimal policy adaptively.

The results confirm the mathematical predictions: at high demand ($L=0.45$), fixed-time control cannot satisfy $L \cdot C < M \cdot x$, causing queues to grow indefinitely. Q-Learning solves this by dynamically extending green time for congested lanes, achieving an 88% reduction in average waiting time.

Future Work

- Variable M: model different discharge rates per lane type.
- Deep Q-Network (DQN): handle continuous L values without discretization.
- Multi-intersection coordination: use multi-agent RL across a signal network.
- Real-time L measurement: use actual sensors to measure arrival rates live.

References

1. [1] Sutton, R. S., & Barto, A. G. Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.
2. [2] Watkins, C. J. C. H., & Dayan, P. "Q-learning," Machine Learning, vol. 8, pp. 279–292, 1992.
3. [3] Gross, D., & Harris, C. M. Fundamentals of Queueing Theory, 3rd ed., Wiley, 1998.
4. [4] Wiering, M. "Multi-agent reinforcement learning for traffic light control," Proceedings of the 17th ICML, 2000.
5. [5] Hillier, F. S., & Lieberman, G. J. Introduction to Operations Research, 10th ed., McGraw-Hill, 2015.
6. [6] Transportation Research Board. Highway Capacity Manual, 6th ed., National Academies Press, 2016.
7. [7] Oracle Corporation. Java SE 17 API Documentation. https://docs.oracle.com/en/java/
8. [8] Chin, Y. K. et al. "Deep reinforcement learning for traffic signal control," IEEE Transactions on Intelligent Transportation Systems, 2023.